Programming Mobile Application 2026

Exam Brief - Draft 1 for Session 8

Table of Contents

- Description of Exam
  - What is a "prototype" in this exam?
  - Practical Information
    - How to Submit Your Exam
    - Generative AI Use
  - Theme of the Application/Prototype
    - Theme Option 1: Open theme
    - Theme Option 2: Select a theme from the list below
- Design, Accesibility and Technical Requirements
  - Minimum Pass Criteria
  - Minimum Design Requirements
  - Minimum Technical Requirements
  - Minimum Accessibility Requirements
- Written Report
  - Written Report Structure
    - Example of a Group Submission
    - Example TOC for an Individual Submission
    - Collaborative Sections
    - Individual Sections
    - Individual Assignment Reflections
  - References
  - Number of pages
  - Advice for the Written Report
- Template Starting Code
- Example of Prototypes with the Minimum Requirements
  - Cat Feeder App
- Checklist for Submission

Description of Exam

The exam requires you to demonstrate hands-on mobile app prototyping through React Native, supported by a written report that documents your technical work, design approach, and key reflections.

Your prototype must include multiple screens, sharing of data across at least 2 of them, and be supported by a Figma design file. The UI and styling of your prototype must be based on this Figma design file, and your code must be compliant with the WCAG 2.2 A level guidelines for accessibility outlined below.

It is **not** expected that your prototype will look *exactly* like the Figma design you've created. However, there should be a clear correspondence between them, and any differences must be reflected upon and explained in your written report.

What is a "prototype" in this exam?

A prototype is a **working**, functional mobile app built with React Native that demonstrates your ability to translate design ideas into actual code. The user should be able to interact with buttons, enter data, and see that data in different parts of the app. Additionally, a prototype is a demonstration of your app's core idea and concept. It transforms the abstract concept in your head into something tangible that users can experience and understand.

A prototype doesn't need to be a fully-featured app. You can have simplified features or incomplete functionality, but you should explain those choices in the written report. The course emphasizes hands-on programming for prototypes rather than design polish or concept perfection, and the exam follows that same emphasis.

Practical Information

- **Exam type:** Pass / Fail
- **Deadline:** Friday 29th of May, 2026
- **Format:** This is a group exam (2–4 students) with an additional individual component.
  - You may also work alone, provided you are registerd as a group with 1 member
- **What you submit:** A `.zip` file containing your code folder + written report as PDF
- **Can I reuse my Assignment 2 code?** Yes, you are welcome to build on it. However, it is not a requirement.

- **Is the concept/idea of my app evaluated?** No. You are evaluated on the minimal requirements set below and your reflection on the process.
- **Minimum Number of Screens required:** 5 app screens + 2 onboarding screens (7 total)
- **How many pages must the written report be?** Check the Number of pages section below

How to Submit Your Exam

How to Submit your Exam via LearnIT

There is an `Exam Activity` on LearnIT. There, one person from your group should submit the exam for all of your group.

File Submission

Each group must submit a `.zip` file containing a folder with the files of the app and the written report as a PDF file. The written report should include a link to the Figma design file on which the code is based.

If you are unsure of how to make a `.zip` file, or your `.zip` file is taking too long to be made/uploaded. Please follow the **file submission guide**

To obtain a link to your figma design, you can follow the guide here.

Generative AI Use

If you use any generative AI tools (such as ChatGPT, Claude, or similar LLMs) in your project, you **must clearly mark** where and how they were used. This includes:

- Specific prompts you provided
- Outcomes generated
- How you integrated and modified the output into your project
- Reflections on the use of these tools

Failing to clearly mark generative AI usage can result in failing the exam on grounds of plagiarism or fraud. For more information, please refer to the Generative AI Guidelines for Students at ITU

For example, if you used Generative AI to add a button you need to say:

- What prompt you used to get that button
- What came out (screenshot of code) and if you modified it
- Reflect on this later on in the reflection

Theme of the Application/Prototype

> *The course and the ILOs clearly state this is a course about **hands-on programming of mobile applications.** You will not be evaluated on the concept of your app/prototype* ***!!***

For your exam, you have two options for choosing a theme. You may also build upon the work you completed for Assignment 2 if you wish, but this is optional. You are also free to start fresh with a different concept.

Theme Option 1: Open theme

> ***OBS****: If you go for an open theme, it's entirely up to you how much time you spend in the ideation/conceptualisation part of your prototype.*

If you choose option 1, the app/prototype can be (almost) anything. It just has to be a prototype of an "app" and conform to the minimal requirements set below.

In practice, this means this can be a health care app, a podcast app, a journal app, a dictionary, pretty much anything.

Though it is open, you have a few restrictions.

These restrictions apply to possible apps that are entirely driven by complex architechtures/ backends that are hard to prototype without at least a bit of functionality in topics we did not cover in class.

For example, You are not allowed to make:

- Games or anything that requires some advanced backend logic
- Any app that requires "real time" collaboration
- Apps for media (audio,photo,video) editing
- A cat/pet feeding app which we built in class

If you are unsure whether your app is feasible, you can always contact the course manager.

Theme Option 2: Select a theme from the list below

If you find the open theme intimidating, pick one of the app themes below for your exam:

- Podcast/Music Player
- Local sports club
- Journal
- To-do / task organiser

Design, Accesibility and Technical Requirements

Minimum Pass Criteria

To pass the exam, you must complete **all** of the following requirements:

- All minimum design requirements listed below
- All minimum technical requirements listed below
- All accessibility requirements listed below
- A written report following the structure described in the "Written Report" section
- The prototype must work

Minimum Design Requirements

You must make a design file that contains all screens in Figma.

Additionally, across the screens the following requirements are set:

- Have three interactive components with variants (Introduced in Session 3)
- These interactive components and their variants must also be coded in React Native
    - In case you weren't succesful in coding them, explain what was challenging about translating the Figma interactivity to React Native.

Minimum Technical Requirements

Minimum Number of Screens

App Screens

- 5 Screens
  - You must use a `BottomTabNavigator` as the main navigation of your prototype
  - One of your screens must also have `NativeStackNavigation`
  - If working in a group, each person must program a minimum of 2 components/screens

Onboardings Screens

> OBS: This section will have slight changes after we introduce the topic in class

- 2 Screens
  - Your onboarding screens should be the first thing that appears when the app is run (Introduced in Session 9 or 10)
  - They must explain to a new user, the logic of the app and how to use it
  - They should include a "Skip" button to go straight to the app

React Native Components and Functions

At least one of each of these components/functions:

- `<View>`
- `<Text>`
- `<Image>`
- `StyleSheet`
- `useState`
- `useNavigation`
- `<Pressable>`
- `<TouchableOpacity>`
- `<TextInput>`
- `createBottomTabNavigator` from React Navigation
- `createNativeStackNavigator` from React Navigation

Your prototype *must* be able to share data/variables across at least two screens.

Minimum Accessibility Requirements

Your prototypes interactive functions **must be fully compliant with the WCAG's 2.2 guidelines at Level A.**

- Provide meaningful alt text for all meaningful images.
- Do not rely on color alone to communicate information.
  - Any status or indicator shown with color must also include text and/or a symbol.
- Use clear page titles and headings so users can understand where they are in the app.
- Maintain a clear heading hierarchy across screens.
- Ensure sufficient color contrast for text (recommended from Level AA, required in this exam).

The rest of the prototype must attempt to be compliant with the same standards, though the minimal requirement is mainly for the interactive components.

Accesibility Implementation Refrence

Use these accessibility props in your code:

Element	Required accessibility props
`<Image>`	`accessibilityLabel`
Main page header `<View>` or `<Text>`	`accessibilityRole="header"`
Any visual/status row that uses color cues	`accessibilityLabel` describing the full meaning
`<TouchableOpacity>` / `<Pressable>`	`accessibilityRole="button"` + `accessibilityHint`
`<TextInput>`	`accessibilityLabel` + `accessibilityHint`
`<Tab.Screen>` in BottomTabNavigator	`tabBarAccessibilityLabel`

Written Report

As part of the exam submission, you must include a written report that reflects on your process and technical work.

- The report must be submitted as a PDF file
- Generative AI guidelines also apply to the written content of the report

Written Report Structure

Example of a Group Submission

```
Cover Page

1. Introduction
2. Background and Ideation
3. Design Rationale and Guidelines
4. Technical Implementation Overview
5. Collective Reflection

6. Individual Contribution: Student A
7. Individual Contribution: Student B
8. Individual Contribution: Student C
9. Individual Contribution: Student D

10. Individual Assignment Reflections: Student A
11. Individual Assignment Reflections: Student B
12. Individual Assignment Reflections: Student C
13. Individual Assignment Reflections: Student D

References (optional)
Appendix (optional)
```

Example TOC for an Individual Submission

```
Cover Page

1. Introduction
2. Background and Ideation
3. Design Rationale and Guidelines
4. Technical Implementation Overview
5. Collective Reflection
6. Individual Assignment Reflections

References (optional)
Appendix (optional)
```

Collaborative Sections

Cover

- Include project title, course name, and exam period.
- List all group members exactly as registered in LearnIT.
- Add one sentence per member describing their main technical contribution.

1. Introduction

Question this section answers: What is your app, who is it for, and what is the prototype meant to demonstrate?

- State the app concept in 2-3 clear sentences.
- Define target users and their main need/problem.
  - In case the app is experimental, explain what the app is trying to achieve and for who.
- Clarify prototype scope: what core functionality is included vs intentionally out of scope.

2. Background and Ideation

Question this section answers: Where did the idea come from, and how did it develop into a concrete prototype concept?

- Explain motivation: why this app idea and why now.

- Show 1-2 references to apps or other media and how they influenced your app.
- Describe how you went from idea and sketches to a Figma UI design
    - If relevant, point out one major change in direction and why you made it.

3. Design Rationale and Guidelines

Question this section answers: Which design decisions did you make, and which guidelines or principles informed them?

- Name the concrete design guidelines/principles you applied.
- Describe the navigation of the different screens in the app and why you set it up that way
- Connect decisions to accessibility/usability where relevant.
- Document trade-offs (what you prioritized, what you simplified, and why).

4. Technical Implementation Overview

Question this section answers: How is the prototype built, and how does it work in practice?

- Show app architecture: navigation structure, key screens, and reusable components.
- Describe one complete user flow step-by-step
    - Example 1. Profile Screen Data Sharing: Go to profile -> Add data -> Show data in homescreen.
    - Example 2. Image generator: Select landscape -> Select which characters to include -> Select time of day -> Generate image -> Show Image.
- Explain how data is passed/shared across screens.
- Include short code screenshots for critical parts (navigation, state, key component logic).
- Mention any deviations from Figma and the technical reason behind them.

5. Collective Reflection

Question this section answers: What did you learn from the process, what challenges did you face, and how would you improve the prototype further?

- Summarize top 3 team learnings from design + coding.
- Describe feedback received (in class or from peers) and concrete changes made from it.
- Explain 2-3 main technical problems you encountered and how you solved (or partially solved) them.

- Acknowledge current limitations of prototype.
- Propose realistic next steps if you had more time.
- How was it to meet the accesibility requirements?

Individual Sections

Each student should include a section with their **name and individual contribution** to the app development. Each person should include:

Question this section answers: What did you personally contribute to the project, and why does that contribution matter?

- Identify the exact components/screens you owned.
- Explain your implementation choices and trade-offs.
- Include at least one annotated code screenshot of your contribution.
- Show how your part integrates with other team members' work.
- Mention any additional contributions (Figma, testing, integration, debugging, coordination).

Individual Assignment Reflections

In addition to the report above, each student must include a **1-page individual reflection** on the two course assignments. This page is submitted in addition to the report and is **not included** in the report page count. The page is structured as follows:

- **Question this section answers:** What did you learn from Assignments 1 and 2, and how did those assignments prepare you for the exam?
- **½ page — Reflection on Assignment 1:** Key skills gained, one mistake/limitation, and what you applied in the exam.
- **½ page — Reflection on Assignment 2:** Key skills gained, one improvement over Assignment 1, and what you would still do differently.

References

You can include references to the academic papers introduced in the course and links to online resources provided throughout the course. However, *since this is a report focused mainly on the code and what you did in this project, references are optional.*

Number of pages

A page is understood as 2400 characters/units per page, including spaces and footnotes. This is the standard page according to ITU.

Each report may be ±10% of the total page count.

Examples of number of pages according to group size

Individual submission:

- **Main written report total**: 7 pages
- **Individual Assignment Reflections**: 1 page total (1 per student, additional)

Total pages submitted: 8 (±10%) distributed as the student sees fit

Group of 2 people:

- **Main written report total**: 11 pages
  - Collaborative sections: 7 pages
  - Individual Sections: 4 pages
- **Individual Assignment Reflections**: 2 pages total (1 per student, additional)

Total pages submitted: 13 pages (±10%)

Group of 3 people:

- **Main written report total: 11 pages**
  - Collaborative sections: 7 pages
  - Individual Sections: 4 pages total (shared across 3 students)
- **Individual Assignment Reflections**: 3 pages total (1 per student, additional)

Total pages submitted: 14 pages (±10%)

Group of 4 people:

- **Main written report total: 11 pages**
  - Collaborative sections: 7 pages
  - Individual Sections: 4 pages total (shared across 4 students)
- **Individual Assignment Reflections**: 4 pages total (1 per student, additional)

Total pages submitted: 15 pages (±10%)

Advice for the Written Report

- **Start documenting early:** Keep notes as you work so writing the final report is easier.
- **Avoid repetition:** Each section should answer a different question (idea, design decisions, implementation, evaluation).
- **GenAI transparency:** Clearly mark any AI-generated text/code and explain how you used and adapted it.
- **Focus on process and reasoning:** Explain not only what you built, but why you made those decisions.

Template Starting Code

If you wish, you can use the template that was given to you for Assignment #2 that includes a `BottomTabNavigator` and 4 screens. You can find it on LearnIT.

Feel free to repurpose any code you've written or seen in the course so far.

Example of Prototypes with the Minimum Requirements

Cat Feeder App

The code for this app without the onboarding is available in Session 9, this is the App we built together in class.

13.13
◀ Code Scanner

Skip

Welcome to Cat Feeding Tracker

Track every feeding in one place so your cat never misses a meal.

13.13
◀ Code Scanner

Skip

Log Feedings Quickly

Add who fed the cat, how much food was given, and optional notes.

13.13
◀ Code Scanner

Skip

Check History Anytime

Open your feeding history and tap an entry to view full details.

13.13
◀ Code Scanner

🐱 Cat Feeding Tracker

Cat Name

No cat name added yet...

⚖️ Cat Weight

4.2 kg

🍽️ Food Today

13.13
◀ Code Scanner

Feed the Cat

1. Person who fed

Enter person name

2. Amount of food (grams)

20

3. Notes / Remarks

Any notes about the feeding

My Profile

Your cat's Name:

Enter your cat's name

Restart Intro Flow

13.13
◀ Code Scanner

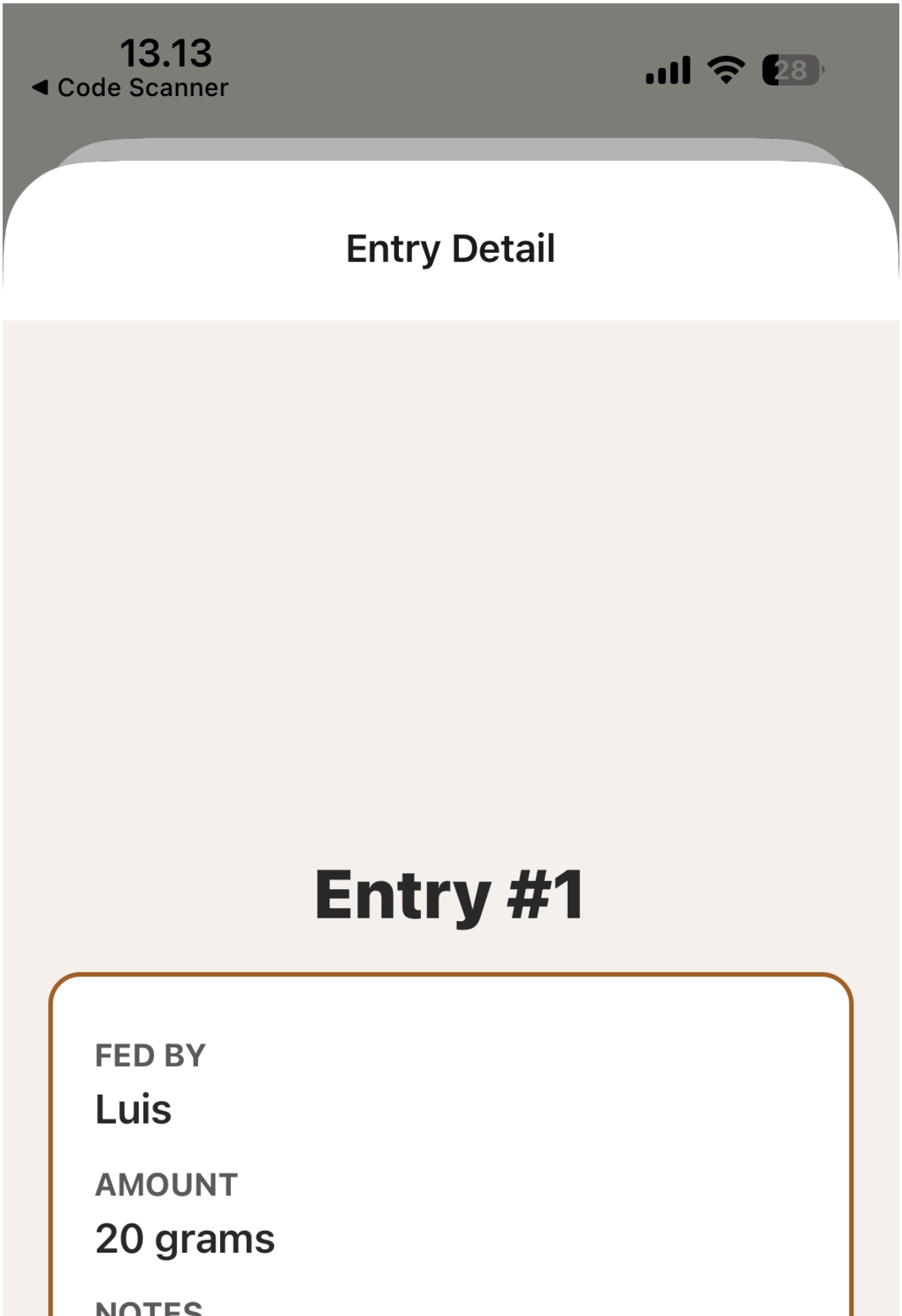
Feeding History

Feeding History

Fed by: Luis

Amount: 20 grams

Notes: Many meows


13.13
Code Scanner
28
Entry Detail
Entry #1
FED BY
Luis
AMOUNT
20 grams
NOTES

Checklist for Submission

Code File Submission Checklist

Here is the checklist for the files that must be submitted for this exam:

- [ ] The folder with your working code
  - [ ] App.js
  - [ ] app.json
  - [ ] assets (folder)
    - [ ] any assets you've added
  - [ ] index.js
  - [ ] package_lock.json
  - [ ] package.json
  - [ ] screens (folder)
    - [ ] all your screen `.js` files

Written Report Submission Checklist

Here is the checklist for the written report that must be submitted with this exam:

- [ ] Written report as a PDF file
  - [ ] Cover page
    - [ ] Title of prototype/app
    - [ ] Group member names
    - [ ] One-line summary of each member's main contribution
  - [ ] Introduction
  - [ ] Background and Ideation
  - [ ] Design Rationale and Guidelines
  - [ ] Link to Figma design file
  - [ ] Technical Implementation Overview
  - [ ] Collective Reflection
  - [ ] Individual Sections (one section per student)
    - [ ] Name and individual contribution
    - [ ] Component(s)/screens coded with code snippets
  - [ ] Individual Assignment Reflections (1 page per student)
    - [ ] ½ page reflection on Assignment 1

- [ ] ½ page reflection on Assignment 2
- [ ] References (optional)